COMPUTER SCIENCE + COMPUTER ENGINEERING

Module 02

Values, decisions and repetition

Four expanded lessons with worked examples, practical assignments and five-question quizzes. Use the main workbook for this module’s project and additional exercises.

4 lessons · 4 demonstrations · 20 lesson questions

Package 02 · Expanded teaching edition · September 28, 2026
Volume 01 of the full undergraduate-to-graduate program
Self-study material. Completion requires implementation, testing and review.

Contents

Use the linked contents or PDF bookmarks to navigate.

Module 02: Values, decisions and repetition 3
Lesson 02.1: Types and expressions 3
Lesson 02.2: Conditions and boundary cases 6
Lesson 02.3: Loops and invariants 9
Lesson 02.4: Functions and contracts 12

Module 02: Values, decisions and repetition

Prerequisites: Module 01.

Integration project: A bounded five-turn resource game. The full brief and review are in the Lab Workbook.

Lesson 02.1: Types and expressions

Learning outcome: Choose integers, real-valued numbers, strings and booleans appropriately.

Choose a representation for the fact

An inventory count describes whole items, so an integer is a natural representation. A player name is text, so it is a string. A door-open flag is true or false, so it is a Boolean. A continuously varying position may use floating-point numbers, which represent a finite set of approximations rather than every possible real number. These are modeling decisions before they are syntax decisions.

The value `12` and the text `"12"` look similar when printed, but their operations differ. Adding integers calculates a numeric sum; adding strings concatenates text. A text prompt produces text even when the user types digits. Decide how the text should be interpreted before combining it with a numeric quantity.

Expressions combine values according to rules

Parentheses make intended grouping visible. Multiplication ordinarily takes precedence over addition, so `2 + 3 * 4` produces 14, while `(2 + 3) * 4` produces 20. Use parentheses to explain a design rule, not just to force an answer by trial and error. The expression `count * price` is meaningful only after both values have passed the required interpretation and validation.

In Python, `/` performs division that can produce a non-integer result, while `//` is floor division. For negative values, floor is not the same as truncating toward zero. Do not choose an operator merely because it printed a whole number in one positive example. Our resource-cost calculation needs multiplication of whole counts, so it does not require an arbitrary rounding rule.

Parse first, then validate meaning

Parsing asks whether a text representation can be converted to the selected type. Validation asks whether the parsed value is allowed by the game. Text `"-2"` can be parsed as an integer, but a negative item purchase is excluded by our contract. Text `"two"` fails the parse itself. Both should lead to a useful message at the interaction boundary, but for different reasons.

`int(text)` either returns an integer or raises an exception for text it cannot interpret. A `try` block identifies an operation whose failure is expected. The matching `except ValueError` branch handles that specific failure. Do not catch every exception and pretend the program succeeded: a coding error elsewhere deserves investigation, not silent conversion to a default price.

Keep the calculation separate from user interaction. The small demonstration below uses fixed strings so that every learner gets the same input. The practical function returns a number or raises `ValueError`; a later command-line adapter will decide how to ask again or display an error. Reusability begins with that separation.

Precision is not solved by rounding everything

Some decimal fractions are not exactly representable in binary floating-point. An equality such as `0.1 + 0.2 == 0.3` can therefore be false. This is not a reason to insert rounding after every operation. Rounding changes the calculation and can accumulate error; first choose the representation and the acceptable comparison criterion.

For exact whole inventory counts and whole game-credit prices, use integers. If a design later introduces currency subdivisions, define the smallest unit explicitly rather than assuming a binary approximation is appropriate. For continuous movement, use a meaningful tolerance when comparing computed results. The mathematical and native-language volumes will revisit representation limits more deeply.

Learn from rejected input

A useful input error tells the player what was expected: “enter a whole-number count” is better than “bad.” Test zero, a normal positive value, negative text, fractional text and nonnumeric text. Zero cost may be legitimate; decide rather than accidentally rejecting it with an overly broad truthiness check.

Worked example

Predict before running `examples/example_02_1.py`.

```
count_text = "3"
price_text = "4"
print("text addition:", count_text + price_text)
try:
    count = int(count_text)
    price = int(price_text)
    if count < 0 or price < 0:
        raise ValueError("count and price must be nonnegative")
    print("numeric cost:", count * price)
except ValueError as error:
    print("Input rejected:", error)
```

The first print deliberately operates on strings and produces concatenated text. The conversion then creates numeric values. Raising a `ValueError` is an explicit refusal to accept a parsed but invalid value; it is not a crash when an appropriate caller handles it. Change one input to `"many"`, then `"-1"`, and explain where the two failures occur.

Expected demonstration output

```
text addition: 34
numeric cost: 12
```

Practical work and pass evidence

Implement `total_cost(count_text, unit_cost_text)` in `labs/student/lab_02_1.py`. Use whole-number text accepted by Python's integer conversion; reject malformed text and negative values with `ValueError`. Return the product without printing. Test count zero, price zero, ordinary values, surrounding spaces, negative values and fractional text.

Run `tools/check_labs.py --lesson 02.1` with your selected Python launcher. Also create a small interaction script that asks for two strings, calls the function and catches only the expected input error. Save an input/output table. Passing the supplied checks is necessary for the function exercise, but you must explain parsing versus domain validation independently.

Changed-case challenge

Add a maximum allowed purchase count of 99 in a separate experiment. Test 98, 99 and 100 without changing the meaning of zero.

Companion source study

[S02] Python: An Informal Introduction; [S04] Python Built-in Types; [S05] Python: Floating-Point Arithmetic; [S08] Python: Errors and Exceptions. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. Why are "12" and 12 different?
2. What does "ab" + "cd" produce?
3. Would an exact item count normally use floating point? Why?
4. Why is `"-2"` a different failure case from `"two"` in a nonnegative item calculator?
5. A programmer fixes text input by concatenating and then converting: `int("3" + "4")`. Does that calculate the sum or product of 3 and 4?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 02.2: Conditions and boundary cases

Learning outcome: Write decisions with complete and non-overlapping rules.

Translate the rule before choosing branches

Our door rule is: a character may enter when it has a key, or when it has both level at least 5 and stamina at least 10. Write the logical structure in words first. The key route is independent of the level-and-stamina route. A key holder with low stamina therefore passes under this particular rule. A different design may choose another rule, but code must implement the one actually specified.

The corresponding expression is `has_key or (level >= 5 and stamina >= 10)`. Parentheses show the grouping directly. Writing `(has_key or level >= 5) and stamina >= 10` changes the rule: now a key holder must also meet the stamina threshold. One familiar positive example may not reveal the difference, so choose a counterexample such as key=true, level=1, stamina=0.

Include the boundary itself

“At least 5” means 5 is accepted. `level > 5` excludes exactly that boundary. Test the value just below, exactly at and just above each threshold. With key/no-key and three values around each numeric threshold, this lesson has 2 x 3 x 3 = 18 nearby combinations. This is a small enough domain to enumerate rather than sample casually.

A truth table gives you independently chosen expected values before you execute the function. Do not generate the expected column using the exact expression being tested; that can reproduce the same mistake twice. Explain each row from the written rule, then use the program to check your explanation.

Choose complete, non-overlapping behavior

An `if`/`elif`/`else` chain selects one branch. Separate `if` statements can select more than one. Suppose one statement says “if stamina >= 10, open” and another says “if level < 5, close.” A character meeting one condition but not the other may trigger conflicting actions. A single decision expression followed by one open/closed result is easier to reason about here.

That does not mean separate conditions are always wrong. Independent actions can be appropriate when a command both records a statistic and plays a sound. The question is whether the actions are alternatives or independent consequences. State the relationship explicitly.

Truthiness is not a substitute for a type contract

A Boolean expresses two states. Other Python objects also have truth values, but that does not mean every nonempty string is an acceptable Boolean input. In particular, the nonempty string `"false"` is truthy. A parser for a human-facing configuration must interpret allowed text forms deliberately.

Our function contract assumes a genuine Boolean key flag and nonnegative integer level and stamina values. Validation belongs at the input boundary in a larger application. This keeps the lesson focused on the logical rule. State the assumption; do not claim the function validates arbitrary untrusted objects when it does not.

Short-circuit evaluation and readable decisions

For `A or B`, Python need not evaluate B once A is truthy. For `A and B`, it need not evaluate B once A is falsey. With pure comparisons that simply saves work. With expressions that also modify state, it can make behavior depend on which parts are skipped. Keep important changes outside a permission expression in this early design.

A readable predicate answers a question: `can_open(...)`. The caller can use its result to choose a message or update a door. The predicate should not secretly spend stamina while deciding whether entry is allowed. That separation will matter when an interface checks permissions repeatedly before the actual action.

Worked example

Predict before running `examples/example_02_2.py`.

```
cases = [(False, 4, 10), (False, 5, 10), (True, 1, 0)]
for has_key, level, stamina in cases:
    allowed = has_key or (level >= 5 and stamina >= 10)
    print(has_key, level, stamina, "->", allowed)
```

The three rows target distinct aspects of the rule: just below the level boundary, exactly at both boundaries, and the key override. Predict each answer from the English rule first. The loop only organizes repeated checks; it does not change permission values between rows.

Expected demonstration output

```
False 4 10 -> False
False 5 10 -> True
True 1 0 -> True
```

Practical work and pass evidence

Implement `can_open` in `labs/student/lab_02_2.py`. Prepare the 18-row table using key=false/true, level=4/5/6 and stamina=9/10/11. Add the key=true, level=1, stamina=0 counterexample. Explain why an expression requiring stamina for every key holder would fail that case.

Run the lesson checks and write one caller that prints an open/closed message based on the returned Boolean. Do not deduct stamina in the predicate. Pass by defending the grouping, the inclusive comparisons and every table row, not just by observing a green test run.

Changed-case challenge

Specify an alternative door rule in which using a key costs stamina. Write the new rule separately; do not silently change the original assignment.

Companion source study

[S06] Python: More Control Flow Tools. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. Does gold > price allow buying an item when gold equals price?
2. Evaluate False or (True and False).
3. Why test immediately below, at and above a threshold?
4. For key=true, level=0 and stamina=0, compare the assigned expression with `(has_key or level >= 5) and stamina >= 10`.
5. Why is a nonempty string such as `"false"` unsafe to use directly as the Boolean key flag?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 02.3: Loops and invariants

Learning outcome: Use repetition that terminates and preserves a stated property.

A loop repeats a rule, not an intention

A loop applies its body according to a defined control condition. A `for` loop over `range(10)` runs with values 0 through 9: ten iterations, not eleven. A `while` loop repeats while its condition remains true. If nothing changes the condition and nothing exits, it may never terminate. “Run until finished” requires a precise account of what makes progress toward finished.

For our resource simulation, the number of turns is an explicit nonnegative integer. A bounded `for` loop is therefore appropriate. Each turn first collects three units and then spends five only when five units are available. The order is part of the rule. Spending before collecting would produce a different trace.

State an invariant you can defend

An invariant is a property that remains true at a stated point during every iteration. Here the resource count is nonnegative after each complete turn. We can defend it in two cases: if there are fewer than five units after collection, nothing is spent; otherwise subtracting five leaves at least zero. Starting at zero establishes the property before the first turn.

For this exact collect-three/spend-five rule, the count after a complete turn is also less than five. Assume the previous count is 0 through 4; after adding three it is 3 through 7. If it is at least five, one subtraction leaves 0 through 2; otherwise it remains 3 or 4. This stronger property depends on our specific numbers and must not be reused unchanged for an arbitrary collection amount.

Distinguish state count from update count

A trace including the initial state has one more entry than the number of turns. Ten updates therefore produce eleven state entries. Zero updates still produce the initial state. This convention makes before/after comparisons straightforward, but it must be stated so that a reader does not mistake the eleven entries for eleven turns.

A sequence of snapshots and a sequence of events are different data. Later, a replay file stores events, while a debugger may store both events and resulting states. Naming them correctly prevents off-by-one errors in playback and testing.

Progress and termination

For a loop of ten iterations, a useful progress measure is the number of iterations remaining; it decreases each time the loop body completes. For an input-validation loop, progress may instead depend on the user eventually entering an accepted value or choosing quit. That is not a guaranteed finite loop unless an explicit retry limit or exit rule exists.

The current assignment has no user-driven stopping ambiguity. Validate the turn count before entering the loop. Negative values should not silently act like zero merely because an empty range results. The contract should distinguish a valid no-work request from an invalid request.

Check the rule with traces and changed parameters

Record resources before collection, after collection, after any spending and after the turn. The final answer alone can hide a wrong ordering. If your implementation spends unconditionally, a case with only three available units exposes the negative result immediately.

Once the fixed rule works, experiment in a separate function with a different collection amount. Reconsider the stronger less-than-five invariant instead of blindly asserting it. This is a first experience with the difference between an implementation-specific property and a more general requirement such as never creating negative inventory.

Worked example

Predict before running `examples/example_02_3.py`.

```
resources = 0
print("initial:", resources)
for turn in range(1, 6):
    resources = resources + 3
    if resources >= 5:
        resources = resources - 5
    print("turn", turn, "resources", resources)
```

The trace repeats a five-turn pattern for the chosen arithmetic, but the implementation should follow the rule rather than hard-code the pattern. The upper range endpoint 6 is excluded, so displayed turn labels are 1 through 5. An `if`, not an unconditional subtraction, protects the nonnegative resource rule.

Expected demonstration output

```
initial: 0
turn 1 resources 3
turn 2 resources 1
turn 3 resources 4
turn 4 resources 2
turn 5 resources 0
```

Practical work and pass evidence

Implement `simulate_resources(turns)` in `labs/student/lab_02_3.py`. Return the initial zero plus the value after every complete turn. Reject negative, non-integer and Boolean turn counts. The strict Boolean exclusion is a schema choice: a yes/no flag is not an acceptable turn count even though Python relates bool and int.

Trace ten turns manually and compare all eleven values. Test zero turns. Explain the nonnegative invariant and show a changed example that breaks an unconditional-spending version. Run the supplied checks, then create at least two additional tests with turn counts not used in the demonstration.

Changed-case challenge

Change collection to seven units per turn in a separate experiment. Explain why a single optional subtraction no longer guarantees a final count below five.

Companion source study

[S06] Python: More Control Flow Tools; [S04] Python Built-in Types. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. How many values does range(2, 7) produce?
2. What must change in while remaining > 0 to make progress?
3. Give an invariant for an inventory limited to ten items.
4. How many entries should the returned trace contain for zero turns and for ten turns under this lesson's convention?
5. Does “resources never become negative” prove that exactly ten turns were simulated?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 02.4: Functions and contracts

Learning outcome: Separate a calculation from interaction and state its valid inputs.

Give one operation an explicit interface

A function names an operation and receives its required inputs as parameters. Calling it supplies arguments for one execution. Our movement function takes position, velocity and elapsed time, then returns a new position. It does not decide how to obtain those inputs or how to display the answer. That makes the same operation usable from a terminal, a test or a future game loop.

A function definition does not perform the calculation immediately. Its body runs when called. Names created inside the call are local to that execution unless a different mechanism is deliberately used. Prefer explicit inputs and returned outputs over hidden global variables in this first design: a test can then describe a complete case without reconstructing unrelated application history.

Write the contract before the implementation

A contract identifies preconditions, the result and permitted side effects. For `advance(position, velocity, dt)`, the inputs must be finite numeric values, dt must be nonnegative and the computed output must remain finite. Negative velocity is allowed. The result is `position + velocity * dt`. The function performs no printing, prompting or mutation of external state.

These choices are part of this lesson's model, not universal laws about all movement systems. A more advanced integrator could deliberately support backward integration; this one does not. A bounded world could reject positions outside a map; this function has no map and therefore makes no such promise.

Return a result instead of showing it

`return` hands a value to the caller and ends that function execution. `print` writes a representation to an output stream. Printing a result does not make it the return value. A function that falls through without an explicit returned result returns `None`, which may cause a later arithmetic operation to fail.

The caller can store a return value, compare it or print it. This separation lets tests check numeric results without parsing terminal text. It also keeps a renderer from having to capture output just to find out where a character moved.

Validate assumptions, not just common inputs

A zero interval should return the original position. A stationary object with zero velocity should remain in place. Negative velocity should move in the opposite direction. Negative time is invalid. Infinity and not-a-number are also invalid for this contract. Even finite input values can produce a non-finite result through overflow, so the reference solution validates the result as well.

The introductory function accepts numeric inputs rather than arbitrary text. The input parser from Lesson 02.1 is a separate boundary. Do not insert a conversion such as `float(anything)` deep inside every function without considering what representations the interface is supposed to accept.

Errors should preserve the caller's choice

On invalid input, raise `ValueError` with a useful explanation. Do not silently clamp negative dt to zero because that hides a caller defect and changes the meaning of the request. A caller may catch the exception, display a message and keep the old state. Assignment happens only after the call returns successfully, which makes this pattern easy to reason about.

A type hint can document intent, but it does not automatically enforce runtime validation. A comment saying “dt is always valid” is not evidence either. Decide which layer checks the precondition and test that boundary.

Revisit the first module with functions

You can now turn the early paper operations into reusable functions: a movement trace, a swap, a displacement calculation and a frame-interval calculation. Their original reasoning remains the same. The wrappers simply make the operations independently callable and testable. Return to the optional starters from 01.1, 01.3 and 01.4 after completing this lesson.

Worked example

Predict before running `examples/example_02_4.py`.

```
import math

def advance(position, velocity, dt):
    if not math.isfinite(position) or not math.isfinite(velocity):
        raise ValueError("position and velocity must be finite")
    if not math.isfinite(dt) or dt < 0:
        raise ValueError("dt must be finite and nonnegative")
    result = position + velocity * dt
    if not math.isfinite(result):
        raise ValueError("new position overflow")
    return result

next_position = advance(2.0, -6.0, 0.25)
print("next position:", next_position)
```

The function does not print; the caller does. A negative velocity is permitted and yields a smaller position. Each validation branch rejects a different unsupported condition before a successful return. Explain what value `next_position` would keep in a caller that already had a value and then caught a failure from a new call.

Expected demonstration output

```
next position: 0.5
```

Practical work and pass evidence

Implement the documented function in `labs/student/lab_02_4.py`. Test normal movement, zero velocity, negative velocity, zero dt, negative dt, infinity, not-a-number and an overflowing result. Capture or inspect output to confirm the function itself does not print.

Write a contract comment naming units and side effects. Then revisit the optional first-module function starters and run their checks. Pass by explaining the difference between returning and printing, and by showing that a caller can retain its old value after catching an invalid update request.

Changed-case challenge

Write a second function that converts milliseconds to seconds and calls advance. Keep unit conversion at this boundary and reuse the original movement function unchanged.

Companion source study

[S06] Python: More Control Flow Tools; [S08] Python: Errors and Exceptions; [S14] Python math library. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What is the result of advance(2, 8, 0.25)?
2. Why is returning a value more reusable than only printing it?
3. What belongs in a function contract besides parameter names?
4. A function prints 12 but has no explicit return. What does `answer = function()` normally bind to answer?
5. Why check the result for finiteness when all three inputs are finite?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.